

보내드림 물류

MSA 기반 국내 물류 관리 및 배송 시스템

TEAM I-이게되네

BONAE-DREAM-LOGISTICS

진혜림 황찬혁 송국희 송채영 문은서 황지호



목차

1. 프로젝트 소개
2. ERD & API 설계
3. 시스템 아키텍처 & 인프라 설계
4. 구현 기능
5. 트레이드 오프 & 트러블 슈팅
6. 잘된 점
7. 회고 및 개선 방향

프로젝트 소개 요약

전국 허브를 거점으로 주문·배송·재고를 관리하는 MSA 기반 B2B 물류 관리 및 배송 시스템

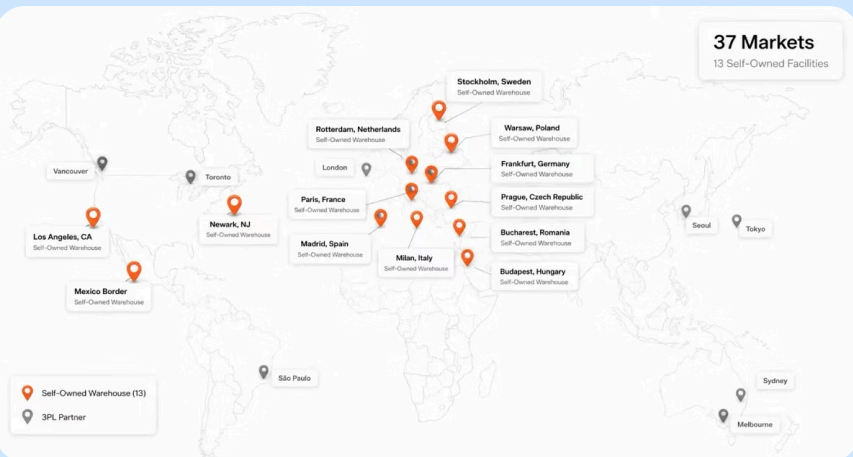
✓ 핵심 도메인

- 사용자/인증 user-service (19001)
- 슬랙 메시지 message-service (19002)
- 허브 hub-service (19003)
- 상품/재고 company-service (19004)
- 주문 order-service (19005)
- 배송 delivery-service (19006)

✓ 주요 기술 스택

- **Backend:** Java, Spring Boot, Spring Cloud
- **MSA:** Eureka, Spring Cloud Gateway, OpenFeign
- **인증/인가:** Spring Security, JWT
- **Database:** PostgreSQL, Redis
- **외부 연동:** Gemini API, Slack API
- **CI/CD:** GitHub Actions, AWS EC2
- **추적/문서화:** Zipkin, Swagger(OpenAPI)

전국 17개 허브를 잇는 B2B 물류 플랫폼



보관

전국 17개 허브에서 다수 업체의 물
품을 보관하고 관리

이동

배송 요청 시 출발 허브에서 목적지
허브로 물품 이동

배송

최종 목적지 업체까지 구간별 경로를 따라 배송 완료

6개 서비스·14개 테이블·서비스별 독립 배포와 확장을 위해 MSA로 설계.
인증, 재고 정합성, 장애 대응까지 실제 운영을 가정.

팀 구성 및 역할분담



진혜림(팀장)

유저·Slack·AI



황찬혁(부팀장)

인프라·인증



송국희

허브·경로



황지호

업체·상품·재고



문은서

주문 관리



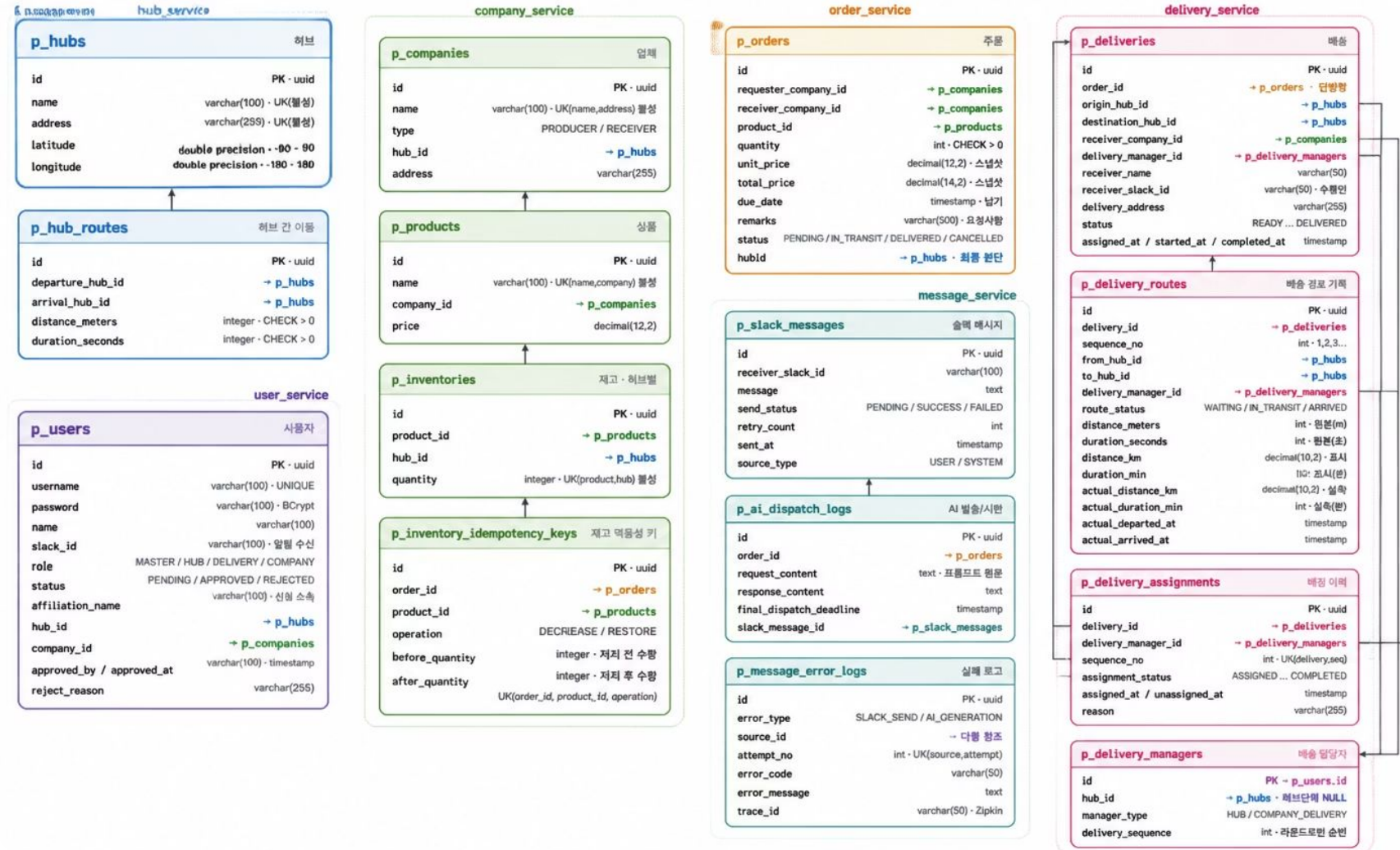
송채영

배송 관리

ERD 설계

보내드림 bonae-dream-logistics — MSA 테이블 관계도 (ERD)

6 services · 15 tables · PostgreSQL / db bonae



경로 설계 — 외부 노출과 내부 통신을 분리하다

클라이언트 요청

/api/{복수형 리소스}

- 게이트웨이 정책 적용
- 외부 요청 처리

외부 요청 → 게이트웨이 → 각 서비스

서비스 간 내부용

/api/internal/**

- 게이트웨이 정책 적용 제외
- 내부 서비스 간 직접 통신

내부 서비스 → 내부 서비스

인증 vs 인가

인증은 Gateway → JWT 서명·만료 검증, 블랙리스트 확인, 사용자 식별 정보 추출

인가는 각 서비스가 담당 → 요청 자원과 사용자 Role/권한을 비교하여 최종 허용·거부

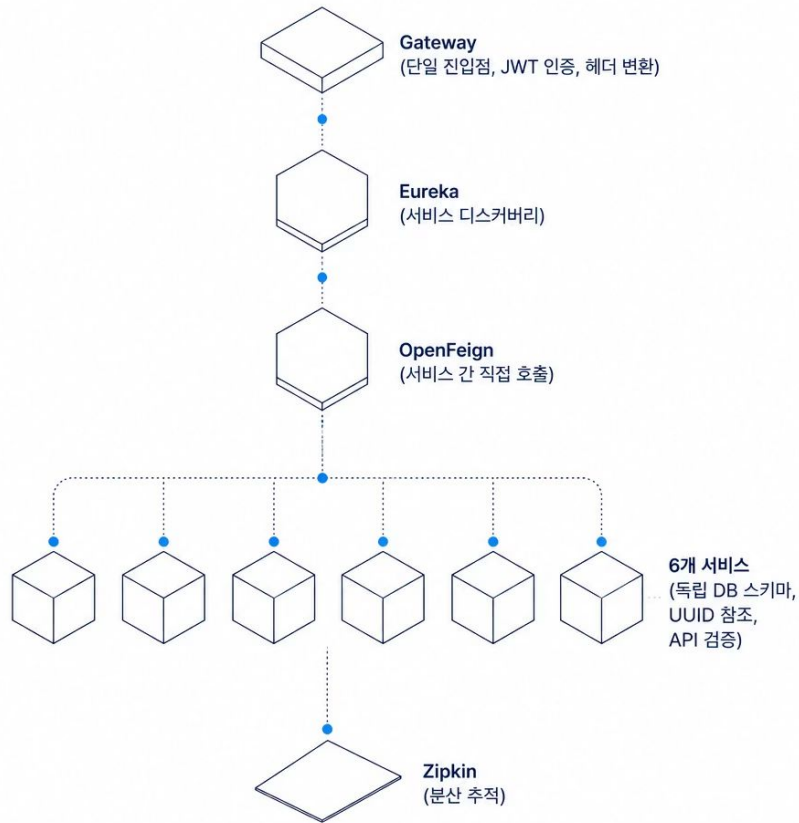
HTTP 메서드

- GET 조회
- POST 생성
- PATCH 수정
- DELETE 논리삭제

헤더 누락 시 에러 응답

X-User-Id / Role 없음
→ **401** (Gateway 미인증 판단)

아키텍처: 단일 진입점과 서비스 분리

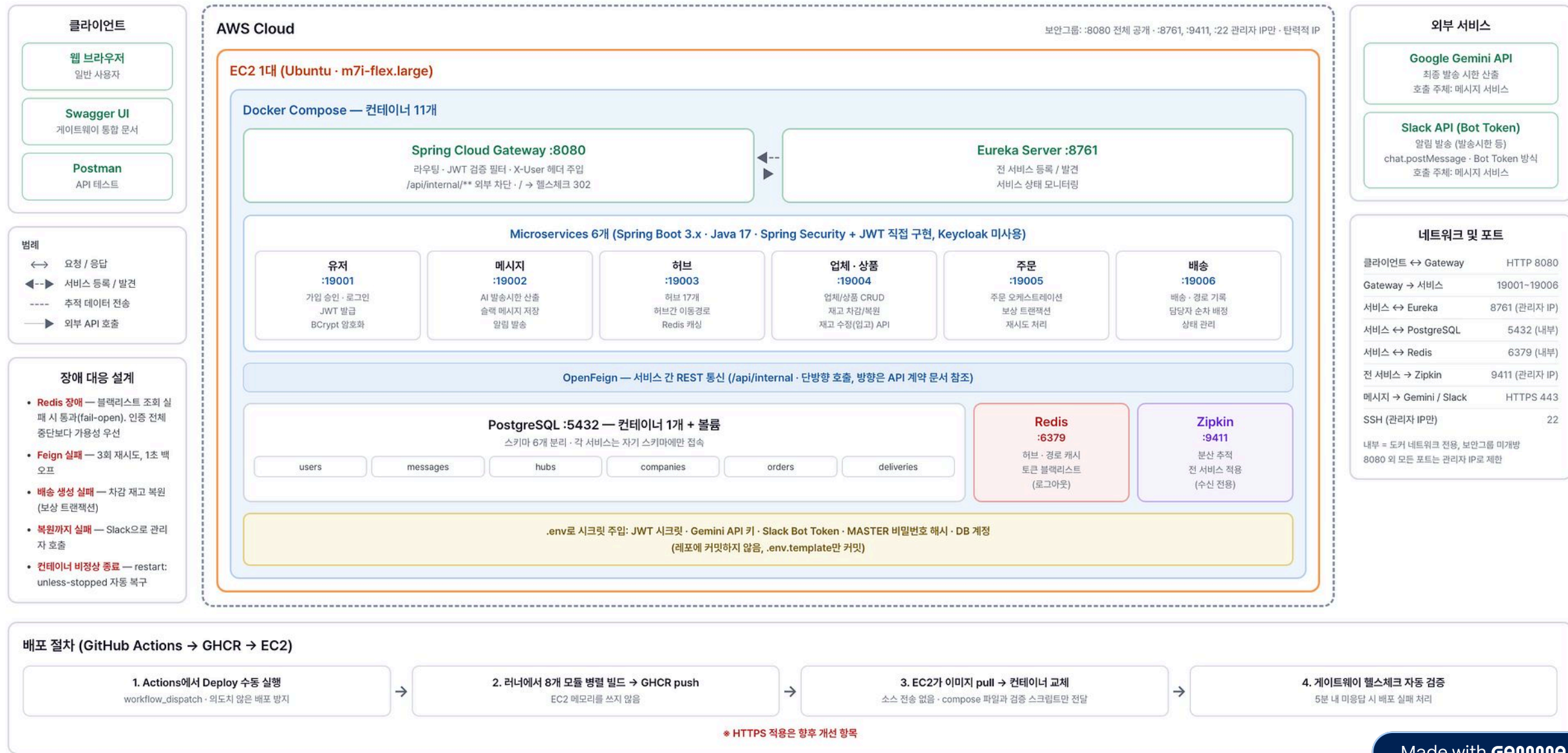


핵심 설계 원칙

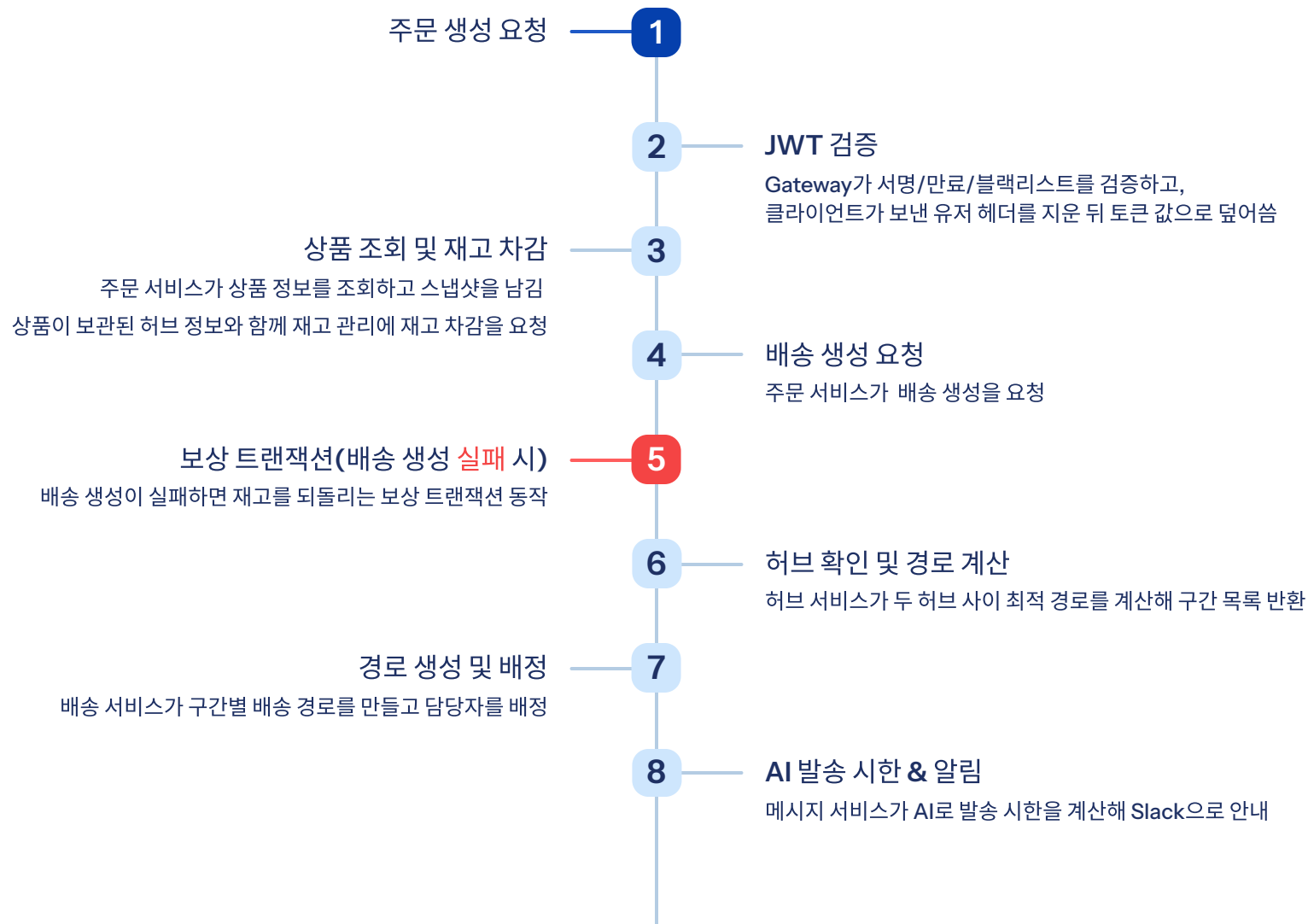
- 외부 요청은 **Gateway** 단일 지점으로만 진입
- 서비스 간 **Eureka** 디스커버리 + **OpenFeign** 직접 호출
- 인증은 Gateway, 인가는 각 서비스 담당
- DB는 스키마 **6개** 분리, 서비스 간 FK 없이 UUID만 보관
- 분산 추적은 **Zipkin**으로 통합

AI 물류 관리 시스템 — 인프라 설계도

MSA 기반 물류 관리 및 배송 플랫폼 · 마이크로서비스 6개 · PostgreSQL 스키마 6개 분리



주문 한 건의 여정 🚀



Redis 장애 대응 전략

보안을 위해 로그아웃 시 블랙리스트를 도입 → 만약 redis 장애가 발한다면?

fail-closed (보안 우선)

- 조회 실패 시 **503**으로 전면 차단
- 로그아웃 토큰 재사용 차단
- 안전하지만 인증이 필요한 모든 API가 함께 멈춤

✓ fail-open (가용성 우선)

- 인증 관련 API만 차단, 나머지는 통과
- 캐시 장애가 서비스 장애로 번지지 않는다는 원칙과 일관
- 장애 기간 동안 로그아웃된 토큰 다시 유효

☐ 결정: MSA의 목적은 서비스장애가 전체장애로 번지지 않는 것 → **fail-open** 을 선택하였습니다.

경로 탐색과 캐시 설계

01. 그래프 기반 최적 경로 탐색

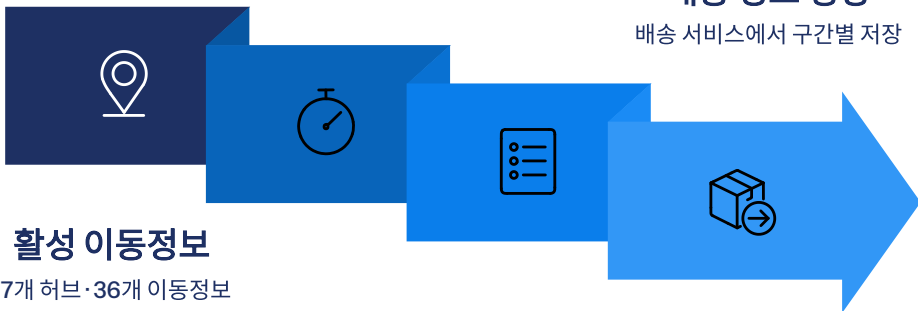
허브 = 정점 · 이동정보 = 간선 · 가중치 = 예상 소요시간

소요시간 기반 탐색

거리보다 도착시간 우선
→ 총소요시간 최소화
현재 규모: 단순 탐색
규모 확대 시: 우선순위 큐 검토

배송 경로 생성

배송 서비스에서 구간별 저장



02. 캐시 설계

대상	키	TTL
허브 상세	허브 ID별	10분
활성 이동정보	단일 키	5분
최적 경로	출발·도착 조합	5분

- 경로 결정에 직접 사용 → 짧은 TTL 5분 적용
- 데이터 변경 시 관련 캐시를 DB 커밋 이후 무효화
- TTL은 무효화 실패에 대비한 안전장치
- 내부 호출 시 캐시 미동작 → 조회 · 탐색 Bean 분리

배송 실행과 분산 장애 대응



구간 상태 전이 검증

이전 구간이 도착해야 다음 구간 출발 허용.
순서를 물리적으로 보장



보상 트랜잭션

배송 생성 실패 시 재고 복원.
서비스 간 호출 3회 재시도. 최종 실패 시
Slack 긴급 알림 + 정합성 이슈 기록



라운드로빈 배정 + 비관적 락

마지막 배정자 다음 순번부터 순환 배정.
조회-계산-쓰기 구간에 비관적 락 적용



중복 생성 이중 방어

앱 레벨 사전 검사 + DB 부분 유니크 인덱스.
제약조건 이름으로 중복 종류를 구분해 도메
인 예외로 변환



AI 발송 시한 + 폴백

AI 계산 실패 시 산술 계산으로 대체.
알림 실패는 배송 생성을 막지 않음

RTR(리프레시 토큰 회전)

보안을 위해 Refresh token을 제시하면 새 **access token**과 새 **refresh token**을 함께 발급하는 토큰 순환 구현

+ 과거 폐기된 토큰이 다시 들어오는 경우 탈취의심 → 저장된 토큰까지 폐기하고 재로그인 유도

```
if (!storedToken.equals(refreshToken)) {  
    refreshTokenStore.delete(username);  
    throw new BusinessException(ErrorCode.REFRESH_TOKEN_NOT_FOUND);  
}
```

6개 서비스가 어긋나지 않은 이유: 계약 우선

코드보다 먼저 **공통 계약**을 정의하고, 회의로 지속적으로 갱신했습니다.

인증 헤더 규격

클라이언트가 보낸 X-User-Id는 게이트웨이가 제거하고 토큰 값 (Id, Role, Hub-id, Company-id)로 덮어씀

내부 API 경로

/api/internal로 통일하고 공개 문서에서 제외

공통 응답 형식 및 페이징 규칙

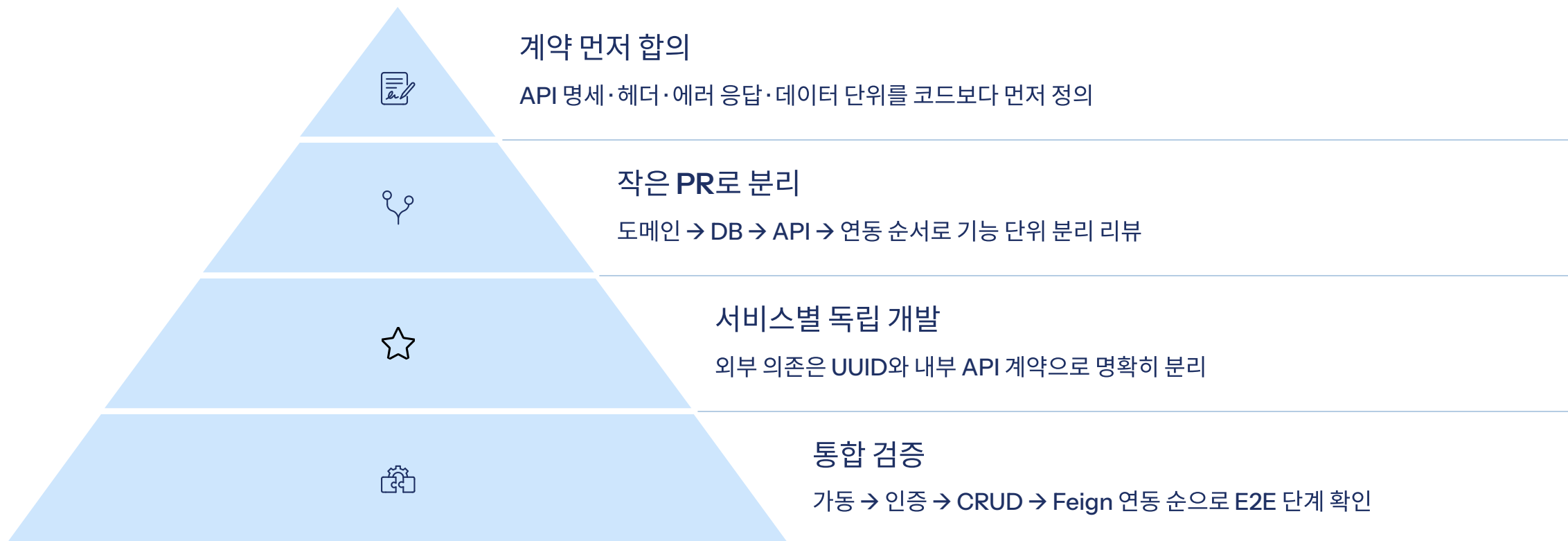
- 공통 응답 형식: 실패는 code, message, traceId, 검증 실패 시 fields 추가. 성공은 DTO 그대로 반환
- 페이징 규칙: 요청은 1-based, 내부는 0-based, size와 정렬 필드는 화이트리스트, 벗어나면 기본값 보정

검증 전략

- 정상 접근(허용)뿐 아니라 비정상 접근(차단)까지 명시
- 실패 사유별로 상태코드를 구분:
토큰 없음·위조 → 401 / 권한 부족 → 403

협업방식

4단계 협업 흐름으로 6개 서비스가 **충돌 없이** 병렬 개발되었습니다.



각 단계는 다음 단계의 기반이 되며, 계약이 흔들리지 않는 한 서비스 간 독립성이 유지됩니다.

추후 개선 사항

1. 보상 트랜잭션 범위 사각지대 해결

- 현재: 배송 생성 실패 시에만 재고 복원처리를 진행
- 문제: `orderRepository.save()` 실패 시 재고 차감이나 배송 생성은 완료됐지만 `Order`는 존재하지 않는 **고아상태** 발생 우려
- 개선: 저장 실패도 보상 대상에 포함하여 배송 취소 및 재고 복원까지 이어지도록 로직 확장

2. 통합테스트 부족

- 코드 구현에 신경을 쓰다보니 통합테스트 시간이 부족
- 열심히 잘 구현 코드에 실제 통합테스트를 많이 못해 아쉬움
- 개선: 통합 테스트도 프로젝트의 하나 과정 → 중간 통합테스트를 할 수 있도록 시간을 마련 후 코드구현

느낀 점

진혜림

사용자 기능을 구현하며 보안의 중요성을 배우고, 팀장을 맡아 협업과 팀워크에 대해서 많이 배우는 시간이 되었습니다. 또 GitHub를 활용한 협업 방식을 익혔습니다.

황지호

개발만큼 설계가 중요하다는 것을 체감하고, 각 기술과 구조의 트레이드오프를 고민하며 상황에 맞는 선택을 하는 경험을 할 수 있었습니다. 시간적 제약으로 더 깊이 고민하고 개선하지 못한 점은 아쉬움으로 남습니다.

문은서

오케스트레이션을 담당하는 서비스인 주문 관리를 구현하면서 계약 문서의 중요성을 많이 느꼈습니다. 다른 서비스와 소통하는 역할을 맡게된다면 더욱 꼼꼼함과 적극적인 커뮤니케이션이 필요할 것 같습니다.

송채영

MSA는 내 서비스 구현만으로 끝나지 않고 서비스 간 계약과 흐름 조율이 핵심이라 다음에는 조연대로 스프린트 단위로 중간 점검을 더 자주 가져가야겠다고 느꼈습니다.

송국희

캐시를 적용하면서 성능 개선뿐 아니라 장애 상황까지 함께 고려해야 한다는 점을 배웠습니다. 특히 Redis 장애가 전체 서비스 장애로 이어지지 않아야 한다는 기준을 세우면서 설계 방향이 명확해졌습니다. 설계 과정에서 선택지와 판단 근거를 기록하는 경험을 통해, 기술 자체만큼 결정의 이유를 남기는 것도 중요하다는 점을 배웠습니다.

황찬혁

인프라가 팀 전체 속도에 영향을 준다는 것을 알게 되었습니다. 통제와 속도, 가용성과 보안 사이에서 트레이드 오프를 정하는 일이 어려웠습니다. 중간 통합의 중요성을 배웠지만, 이를 언제 마일스톤 잡아야하는지는 여전히 과제로 남았습니다.

🙏 경청해 주셔서 감사합니다. 🙏

